Homework 2: Mesh Edit

CS184/284A Computer Graphics & Imaging — Spring 2026

Cedric Chan

🌐 Live Webpage 💻 GitHub Repository

Table of Contents

Section I: Bezier Curves and Surfaces

Part 1: Bezier Curves with 1D de Casteljau Subdivision

Part 2: Bezier Surfaces with Separable 1D de Casteljau

Section II: Triangle Meshes and Half-Edge Data Structure

Part 3: Area-Weighted Vertex Normals

Part 4: Edge Flip

Part 5: Edge Split

Part 6: Loop Subdivision for Mesh Upsampling

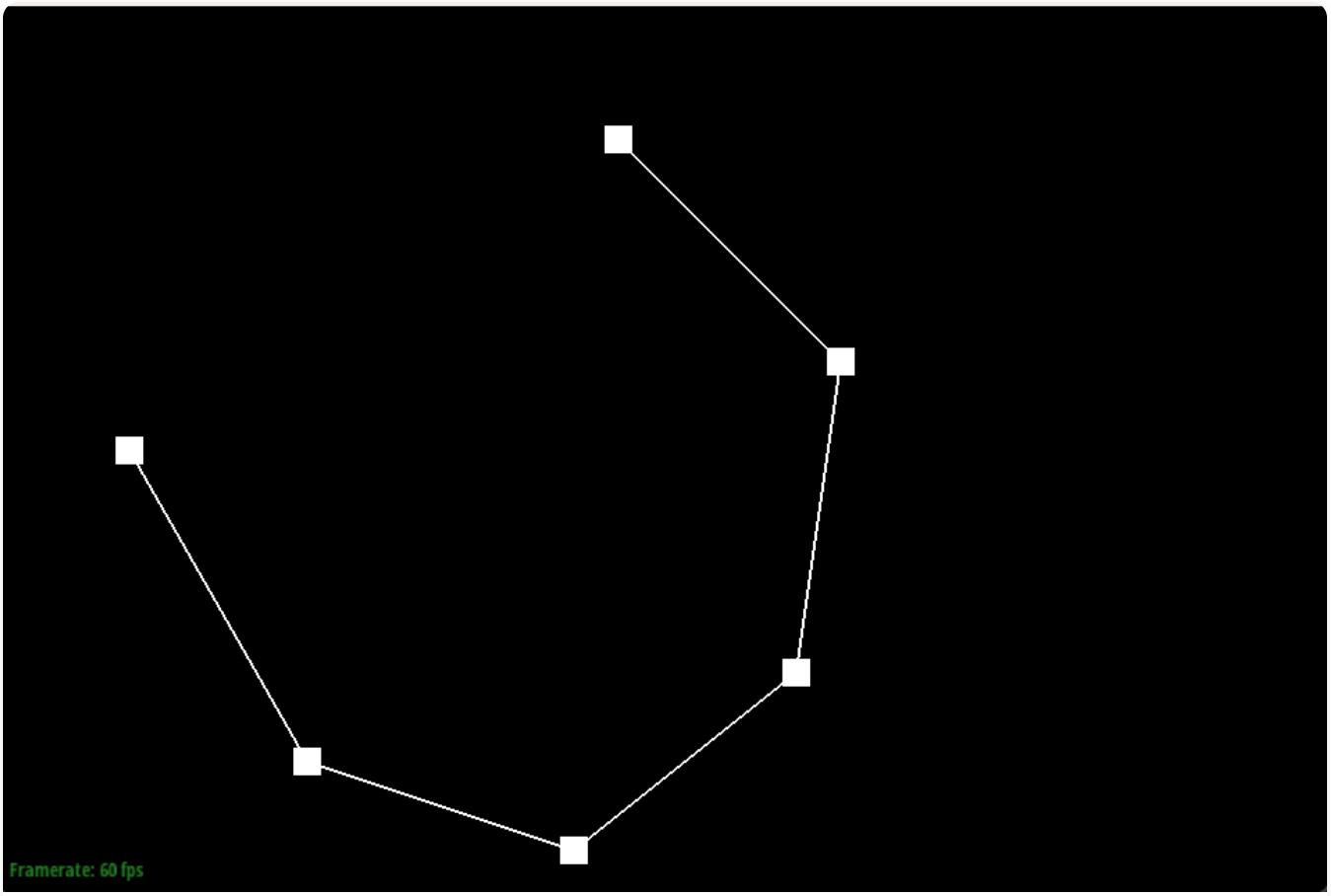
Overview

This homework taught me the basics of 3D representation in graphics. Homework 1 was about rendering and rasterizing 2D images — the algorithms and math used in 2D would be horribly inefficient if naively expanded to 3D. I found it fun and interesting to work with meshes and

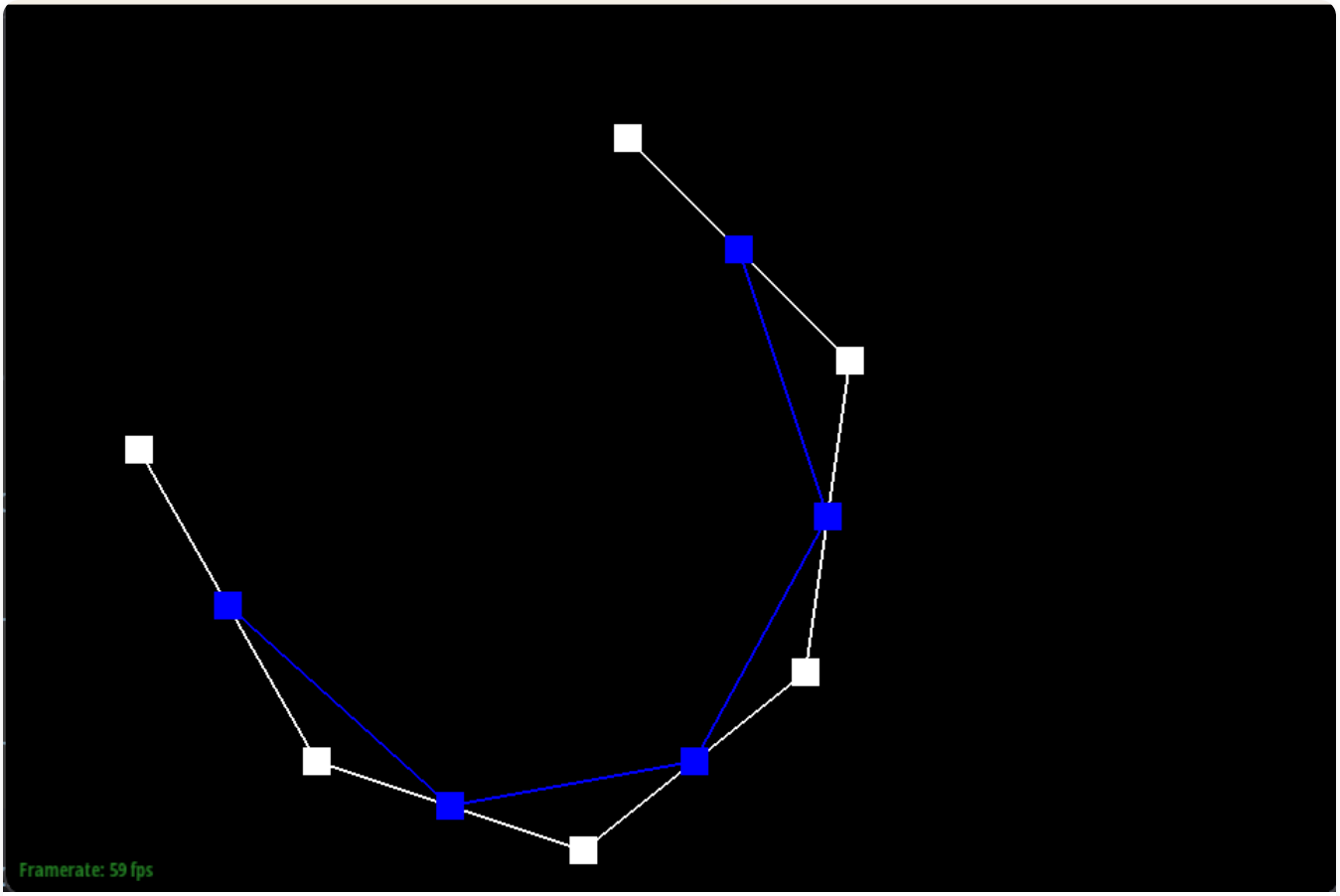
Bezier curves to understand the speedups and efficiencies that come with different methods of 3D representation in graphics.

Part 1: Bezier Curves with 1D de Casteljau Subdivision

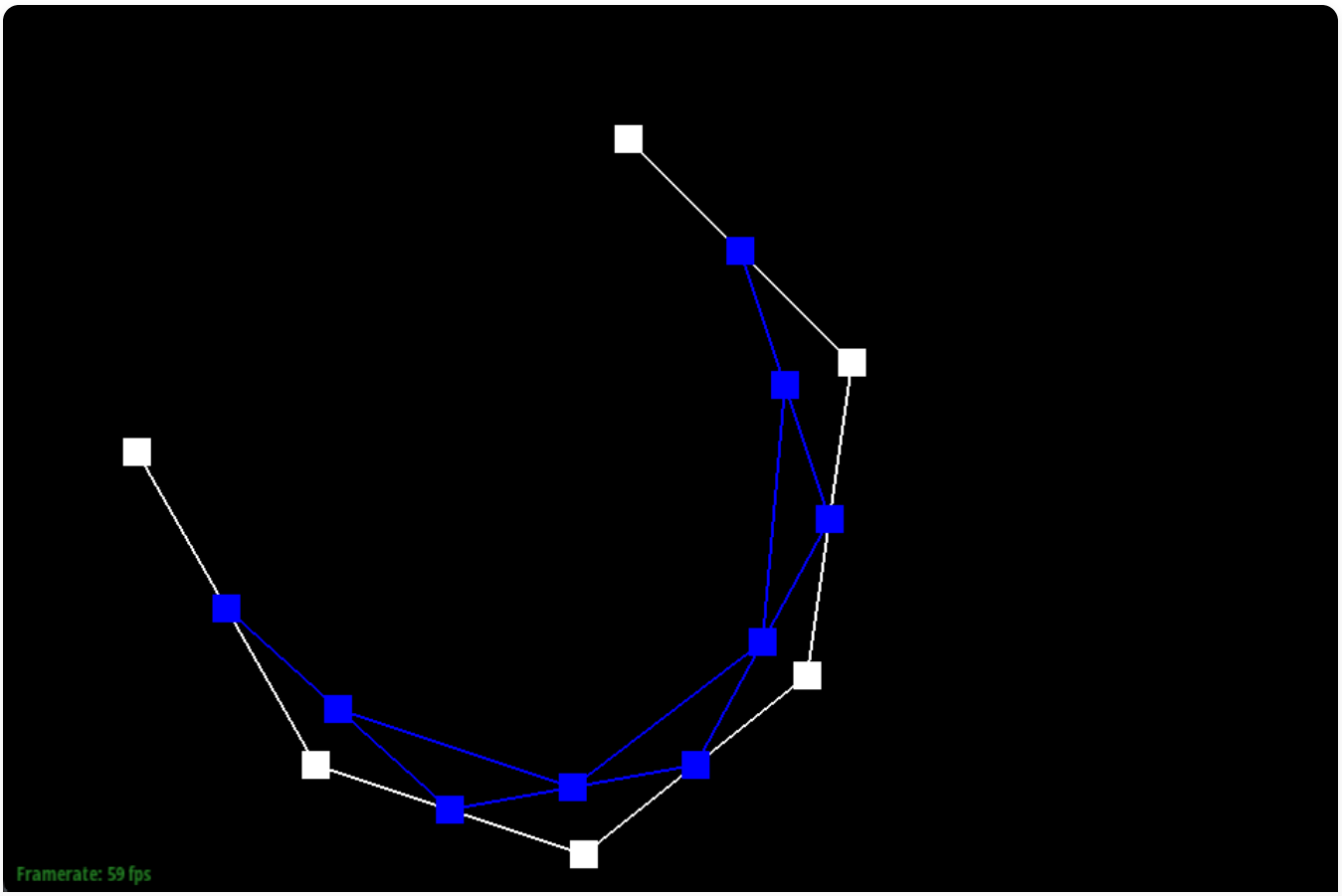
To me, de Casteljau's algorithm is a way to calculate the value of a Bezier curve at a certain point. Each step finds the intermediate control point by linearly interpolating between two neighboring control points. Rinse and repeat until we reach a single point — and this point lies on the Bezier curve. I implemented the algorithm exactly as described in the spec.



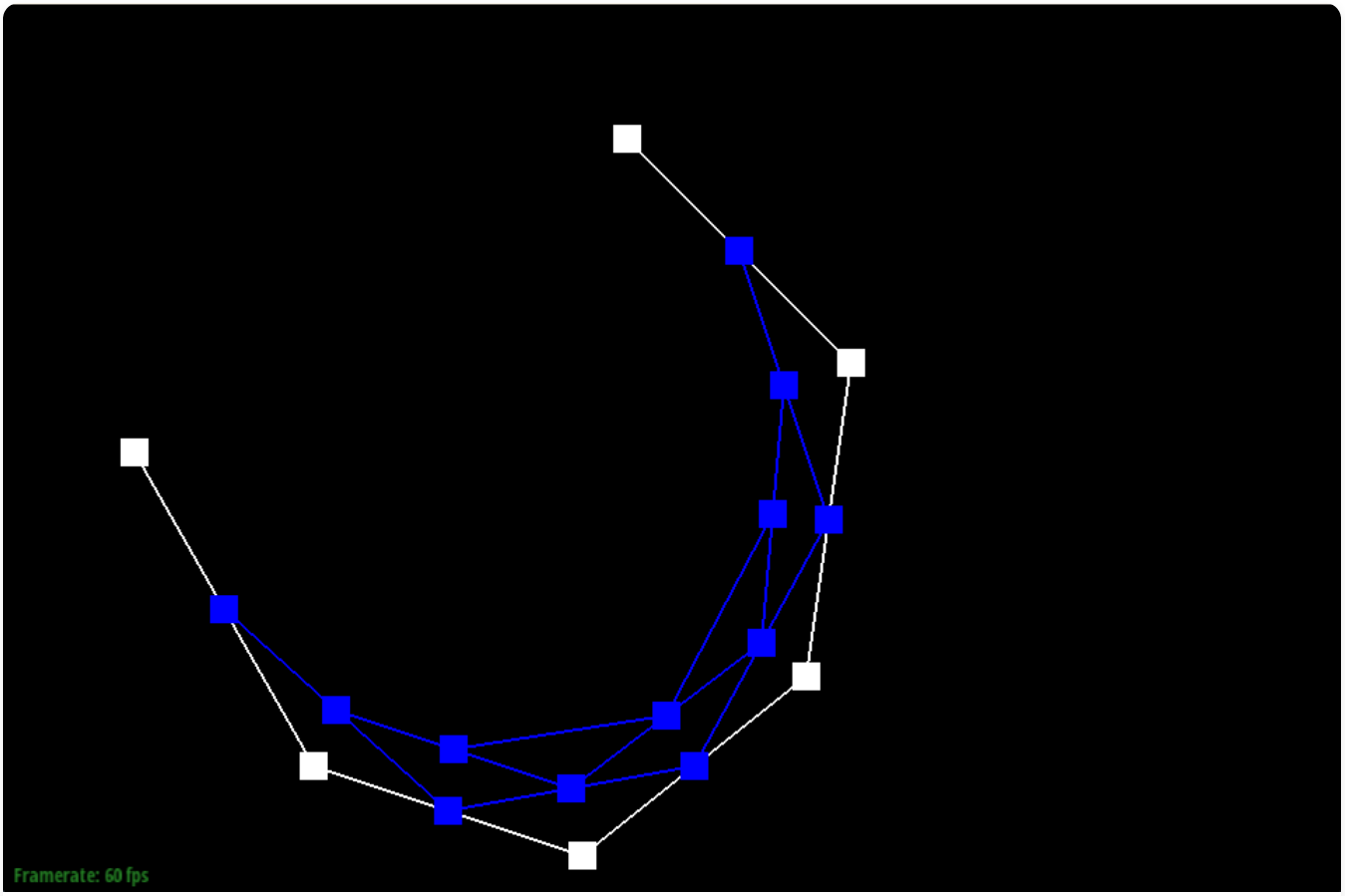
Step 1



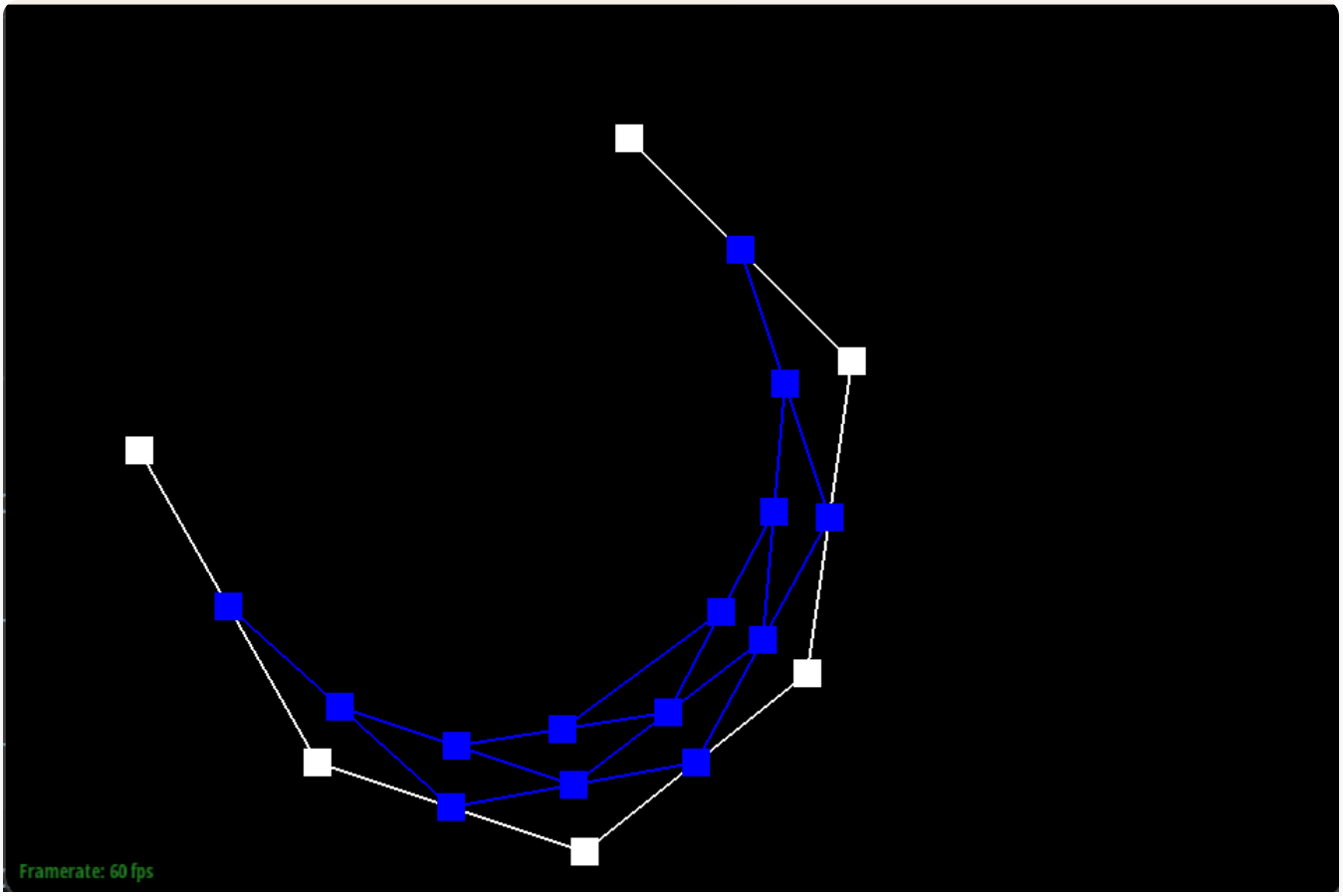
Step 2



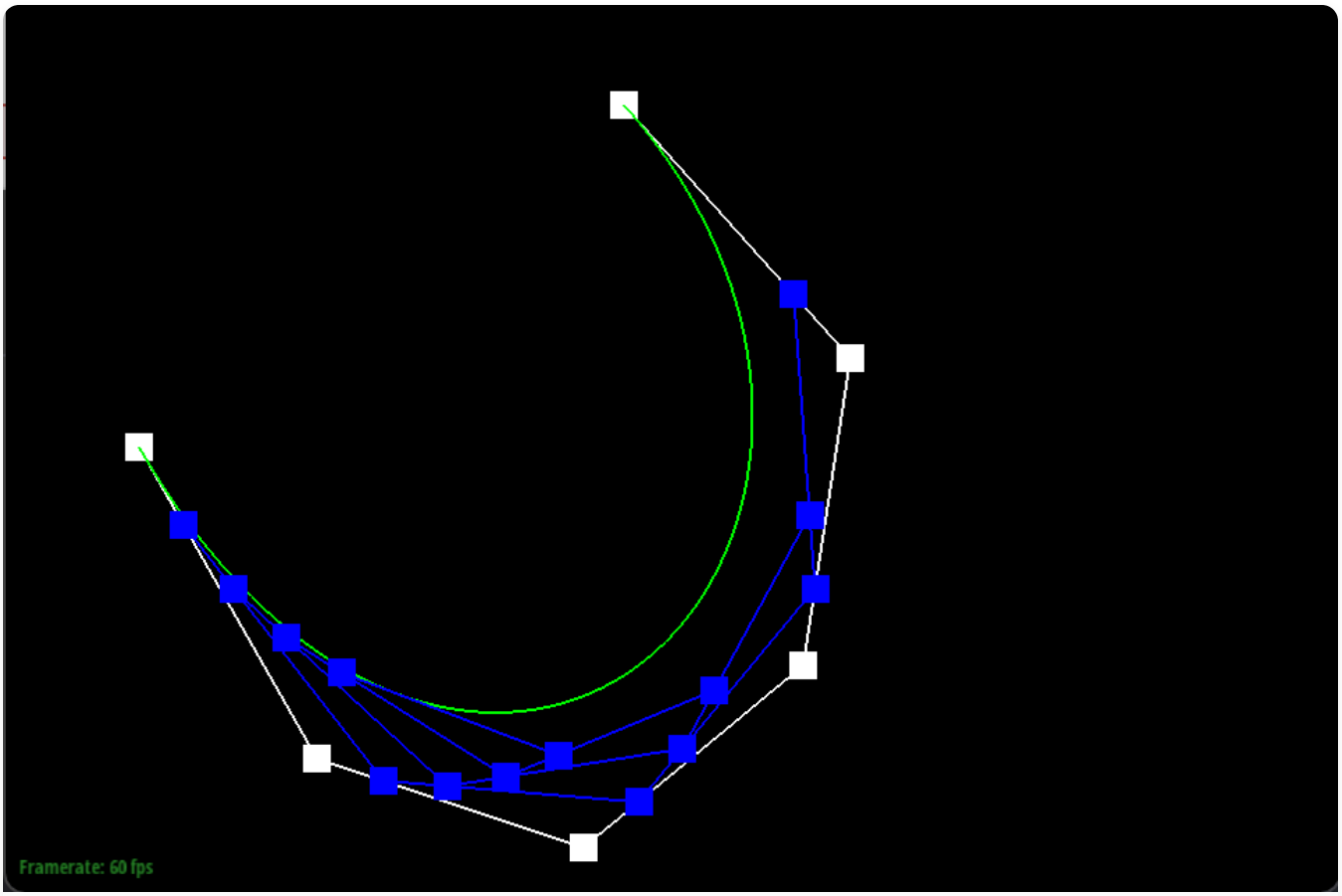
Step 3



Step 4



Step 5



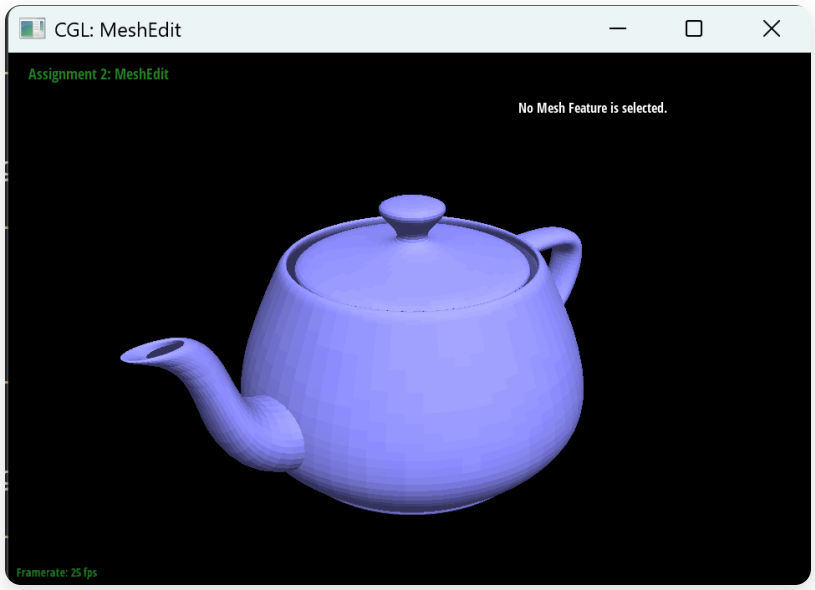
Final curve

Part 2: Bezier Surfaces with Separable 1D de Casteljau

In Part 1, I implemented de Casteljau's algorithm for 2D Bezier curves. In Part 2, we extend to 3D Bezier surfaces by applying the algorithm twice — once in each direction.

A Bezier surface is defined by an n×n grid of control points. Given parameters (u, v), you first run the full 1D de Casteljau along each row using u, collapsing each row of n points down to a single point. Then you run it one more time on those n points using v to get the final surface point.

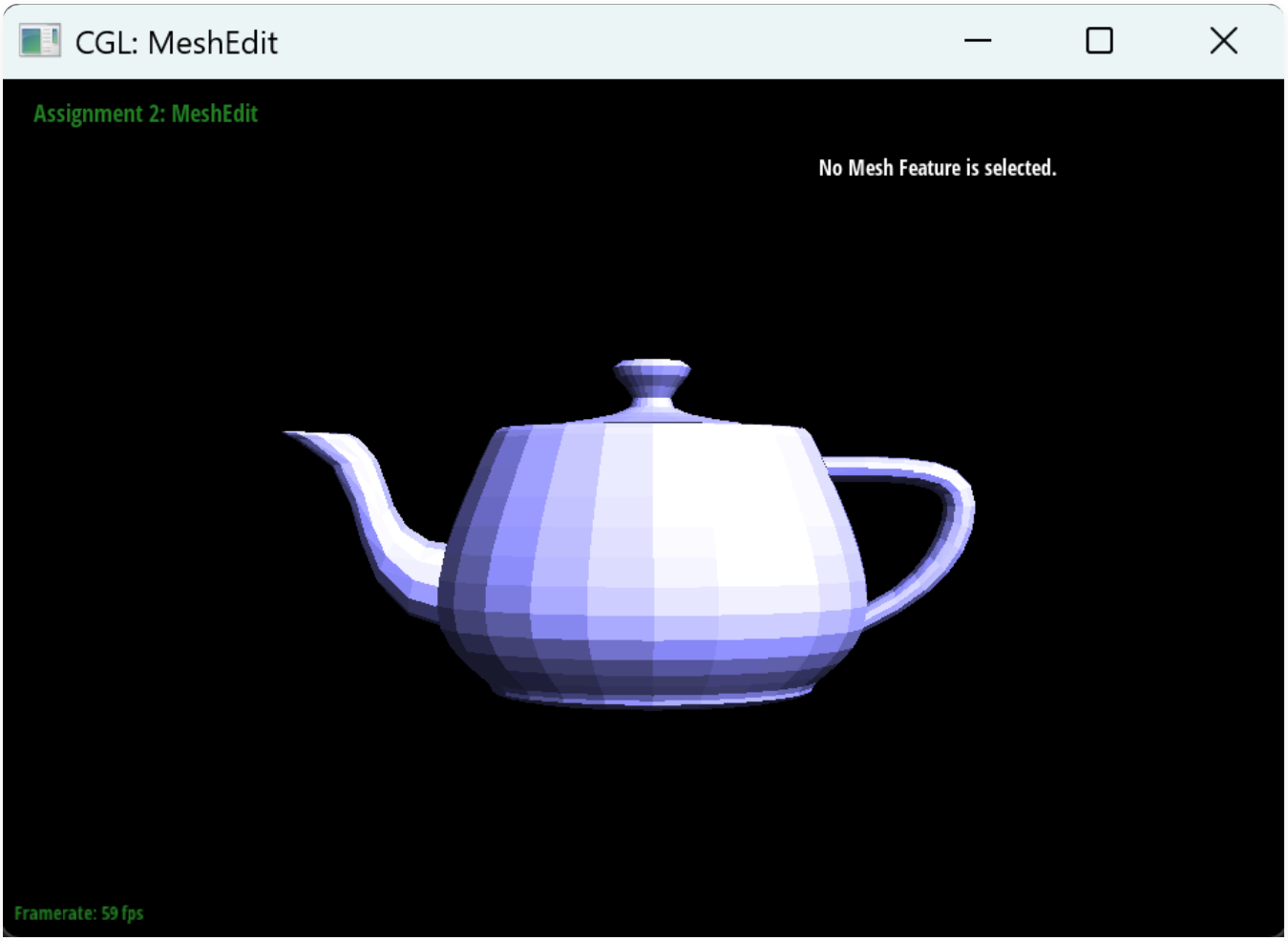
In code, `evaluateStep` is nearly identical to the curve version. `evaluate1D` repeatedly calls `evaluateStep` until one point remains. `evaluate` loops over each row, calling `evaluate1D` with u to collapse each row, then calls `evaluate1D` one final time on those results with v.



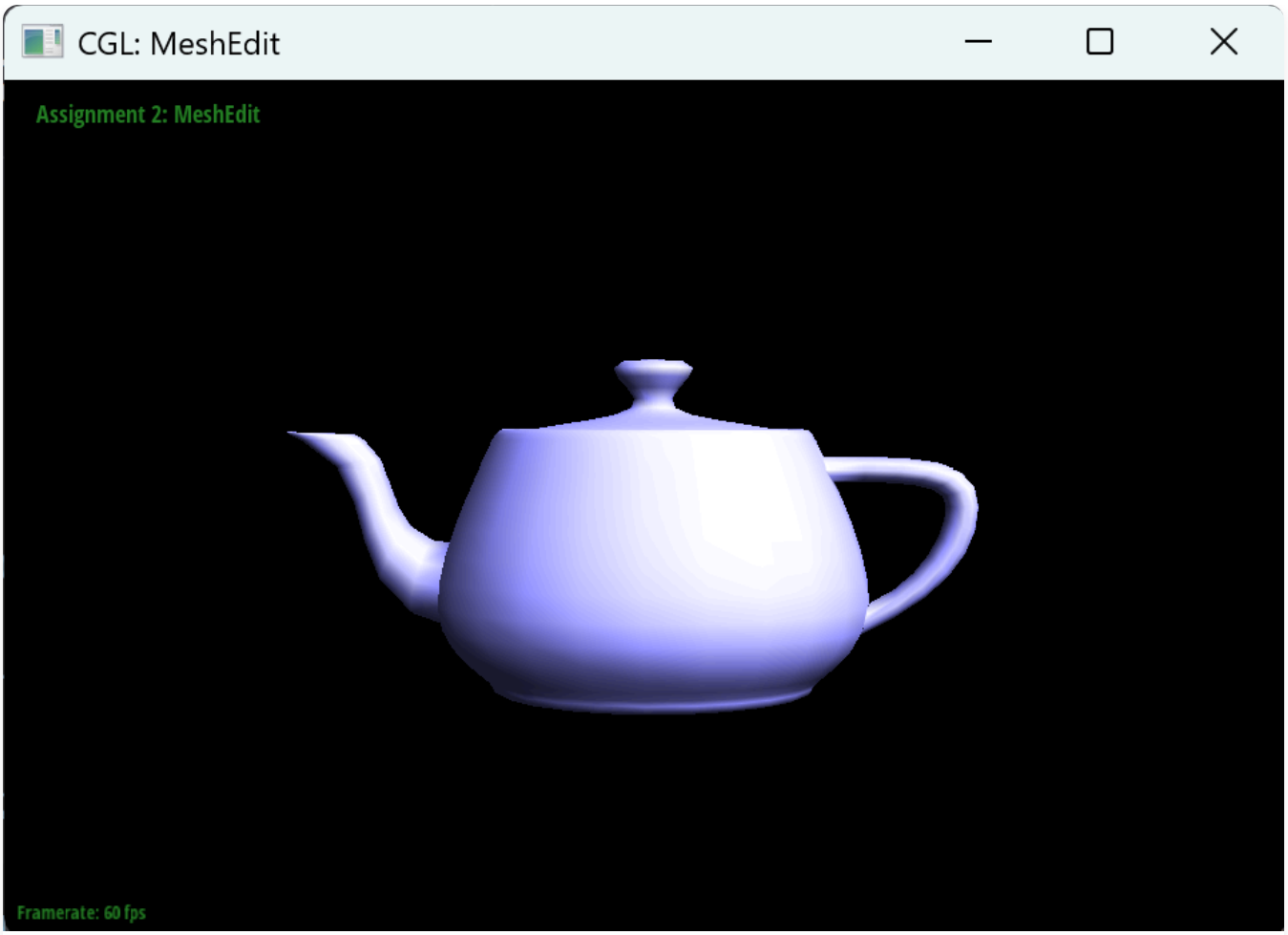
Teapot rendered with Bezier surfaces

Part 3: Area-Weighted Vertex Normals

To get the area-weighted vertex normal, I looped over all neighboring faces of the vertex using the half-edge structure. For each face I grabbed two edge vectors going out from the vertex and took their cross product. The cross product gives you both the normal direction and the area at once — its magnitude is proportional to the triangle area, so area-weighting comes for free. I accumulated all those cross products into a sum and normalized at the end.



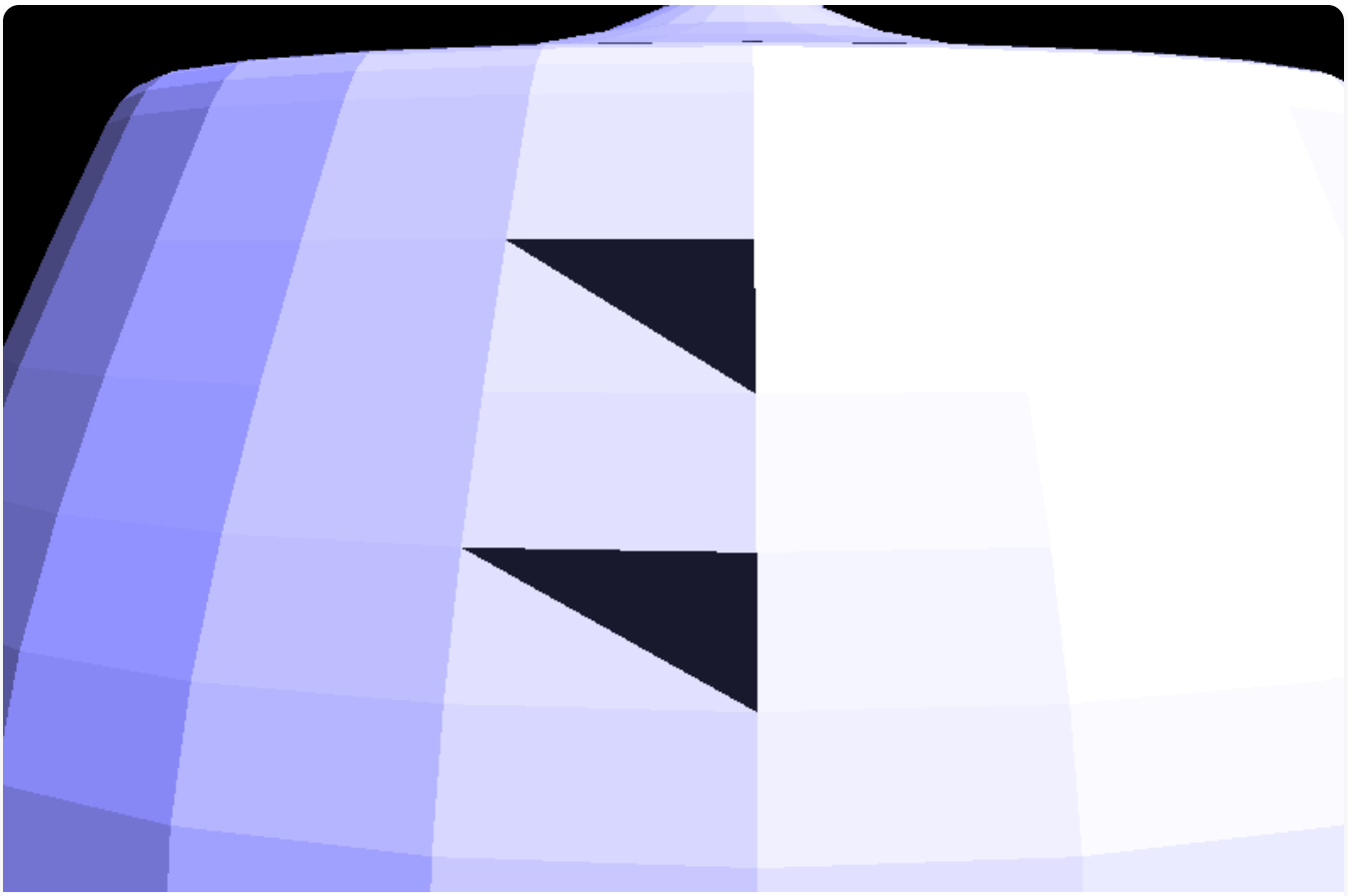
Before: flat shading



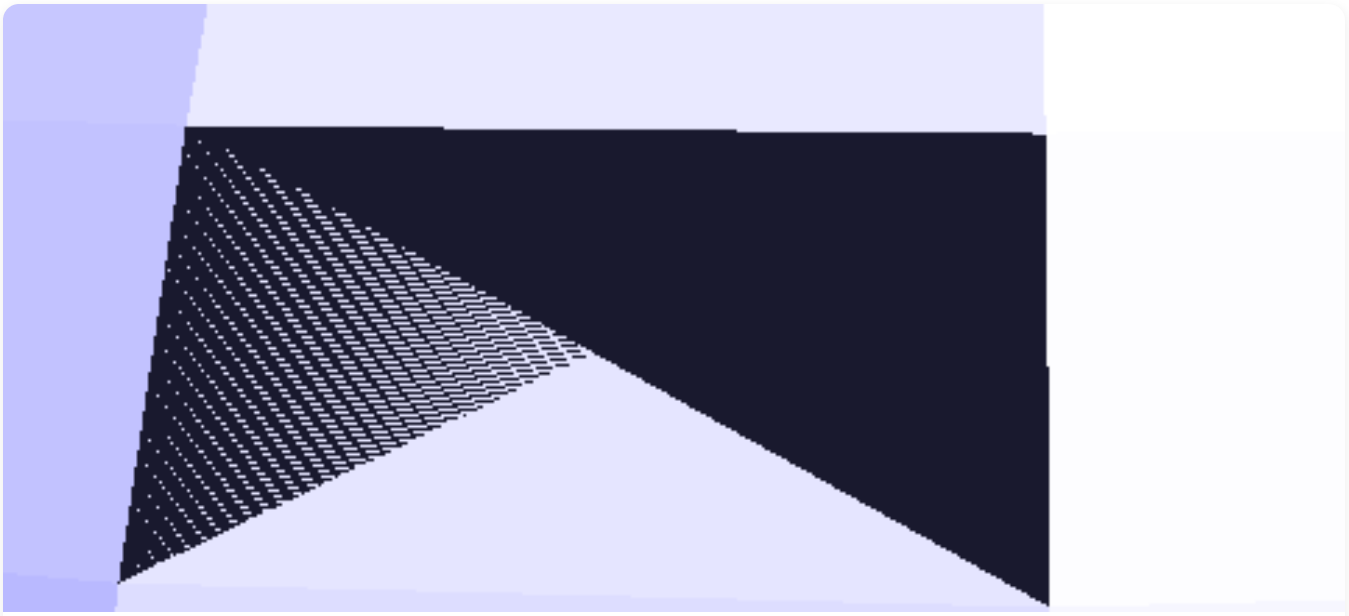
After: Phong shading with area-weighted normals

Part 4: Edge Flip

Okay, I'll be honest here. Initially I was in a time crunch to finish by the deadline, so I was throwing the code into Claude and GPT to try and get them to converge on a correct answer. But the implementations kept having stupid bugs where one triangle would be black, as shown below.



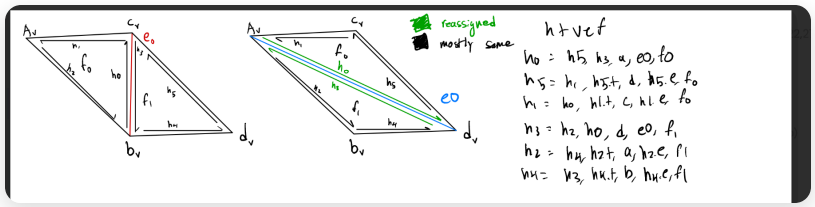
Bug 1: black triangle



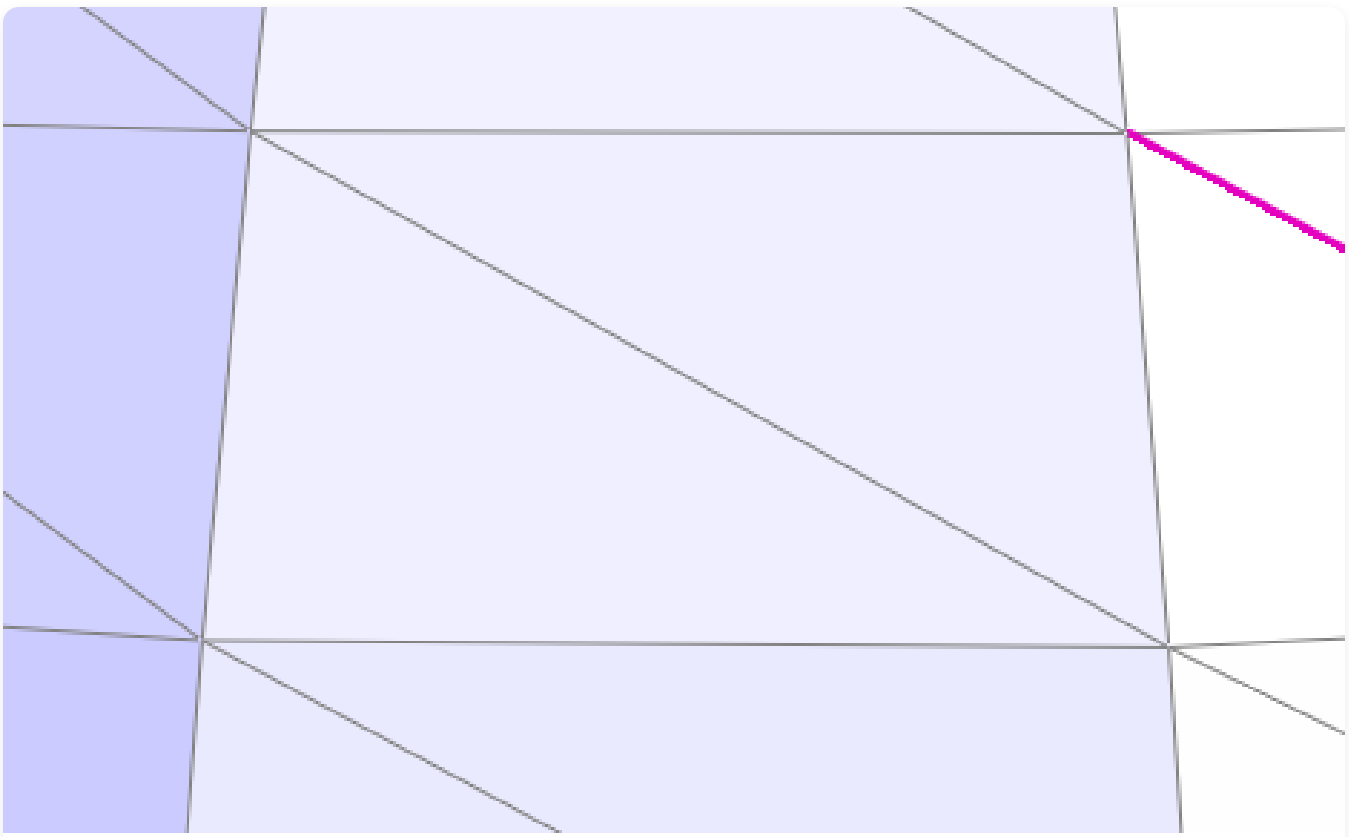
Bug 2: another black triangle

And as you do, I would feed in the code and have it output debug statements to see what was going on. But the models kept going "I FOUND THE BUG" then would reason and say "wait I

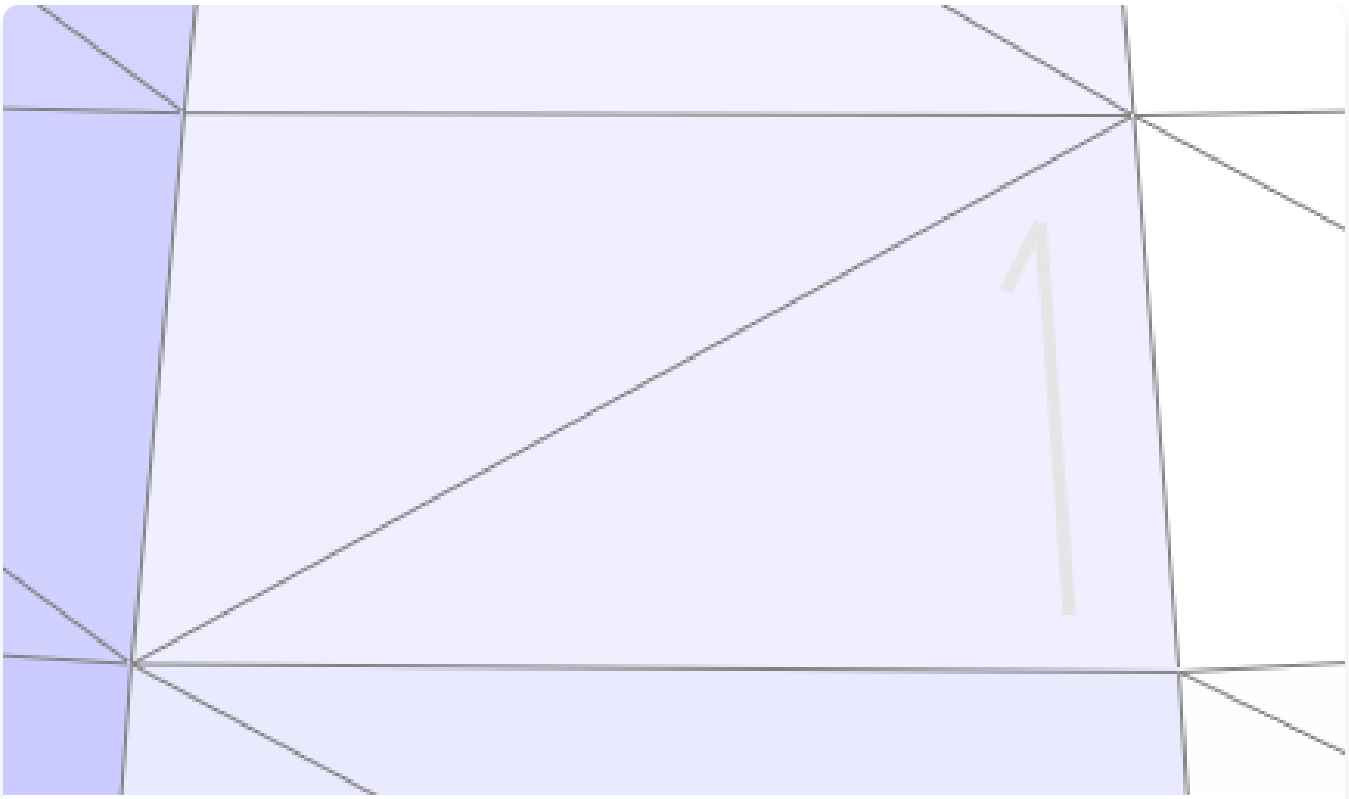
need to backpedal" and loop like that for 2 hours. So I just drew the entire thing out, coded it myself from scratch, and it worked first try. Lowkey shat my pants a bit but I learned a valuable lesson.



Hand-drawn diagram of the flip algorithm



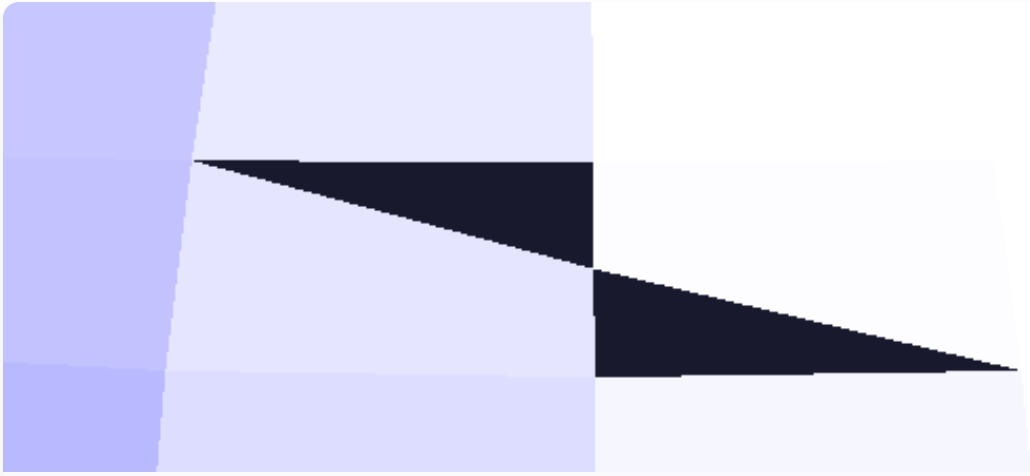
Before flip



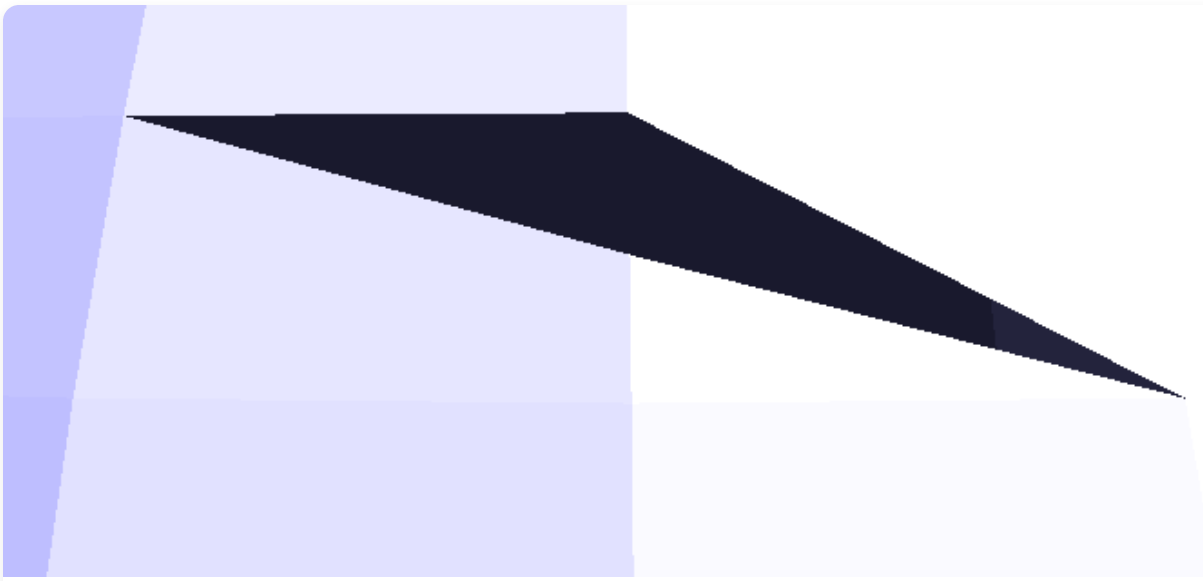
After flip

Part 5: Edge Split

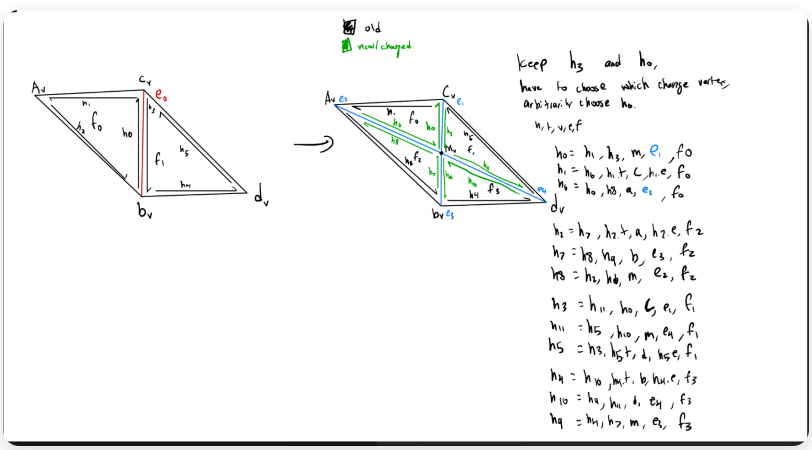
In a very similar story to flip, I was implementing both simultaneously, ran into a bug, drew it out, and had it work first try.



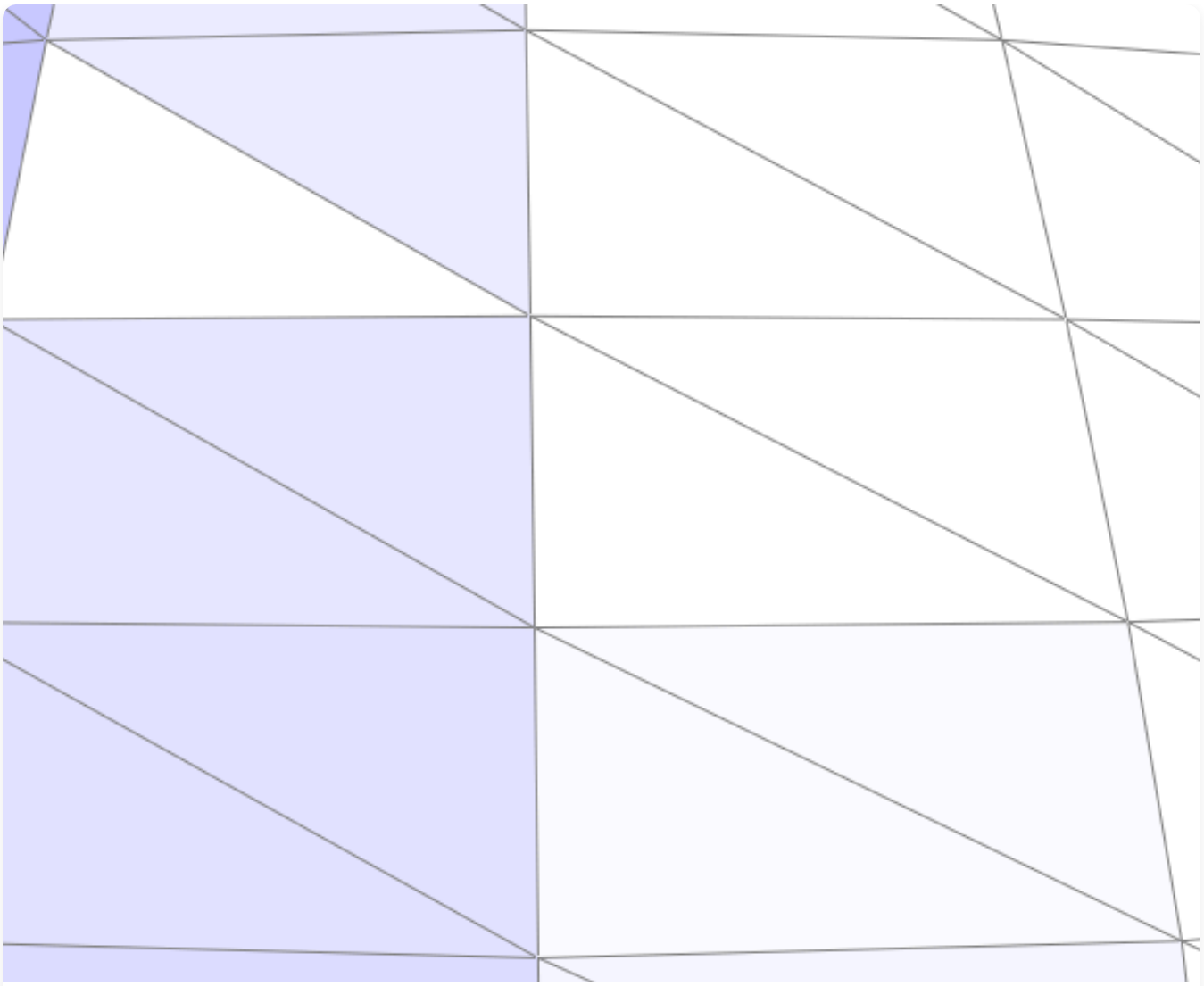
Bug 1



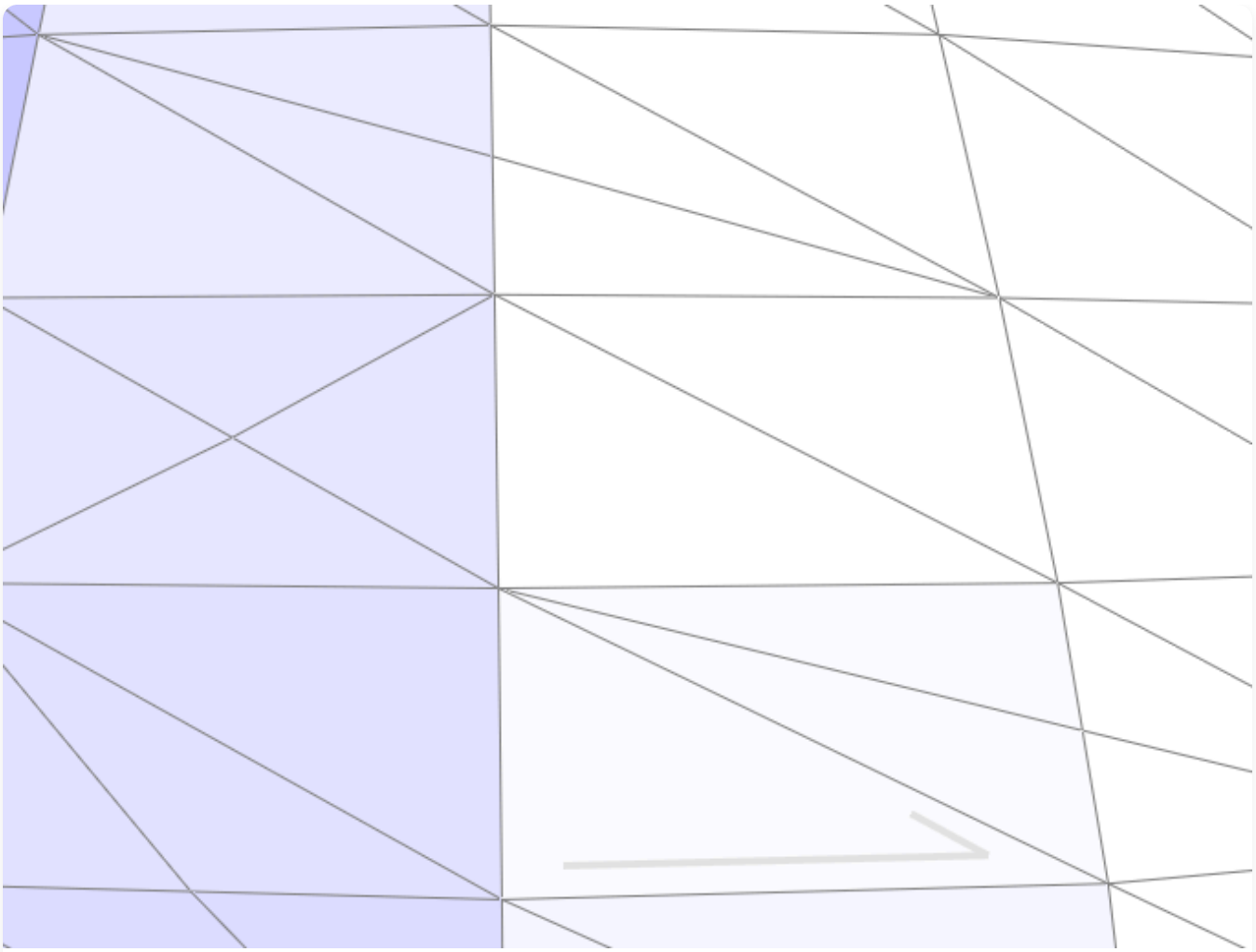
Bug 2



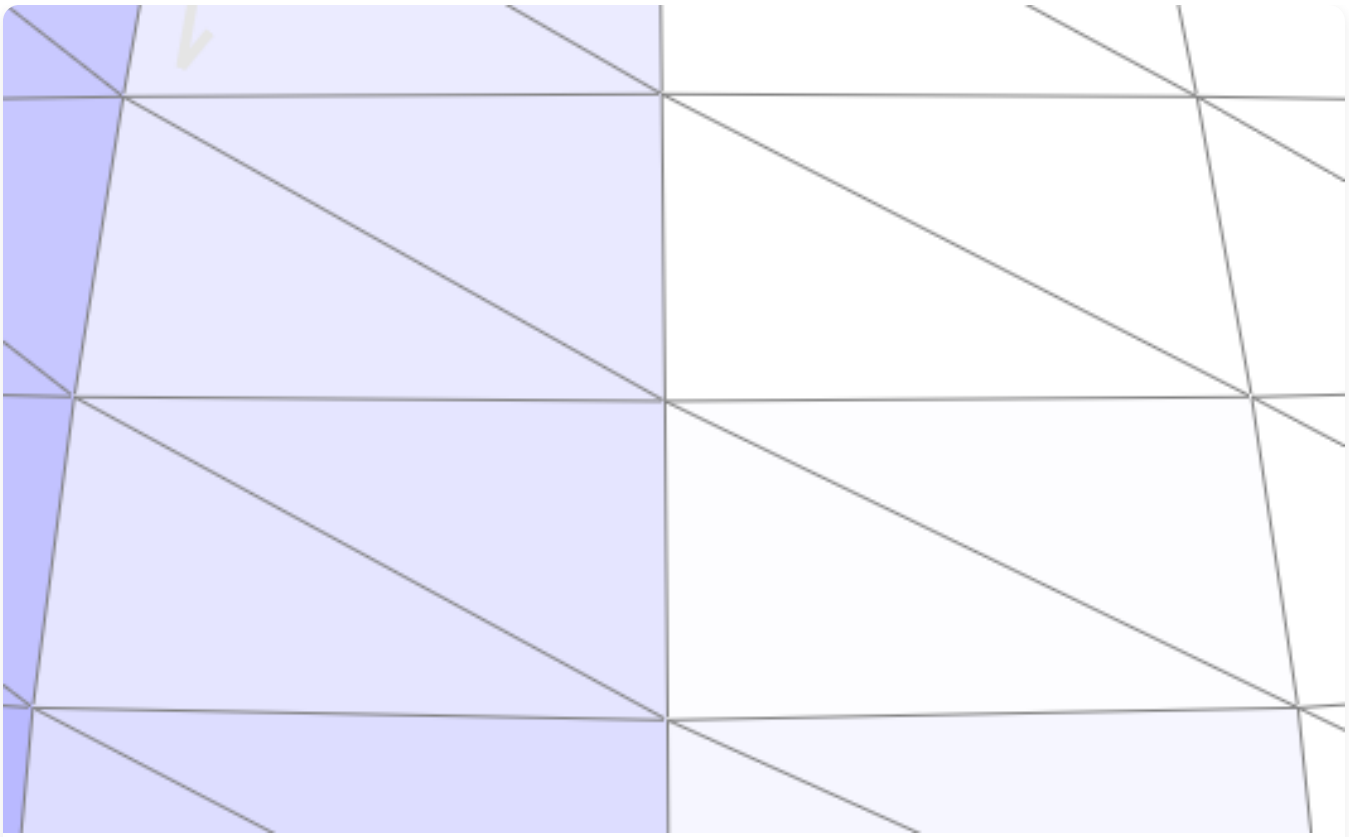
Hand-drawn diagram of the split algorithm



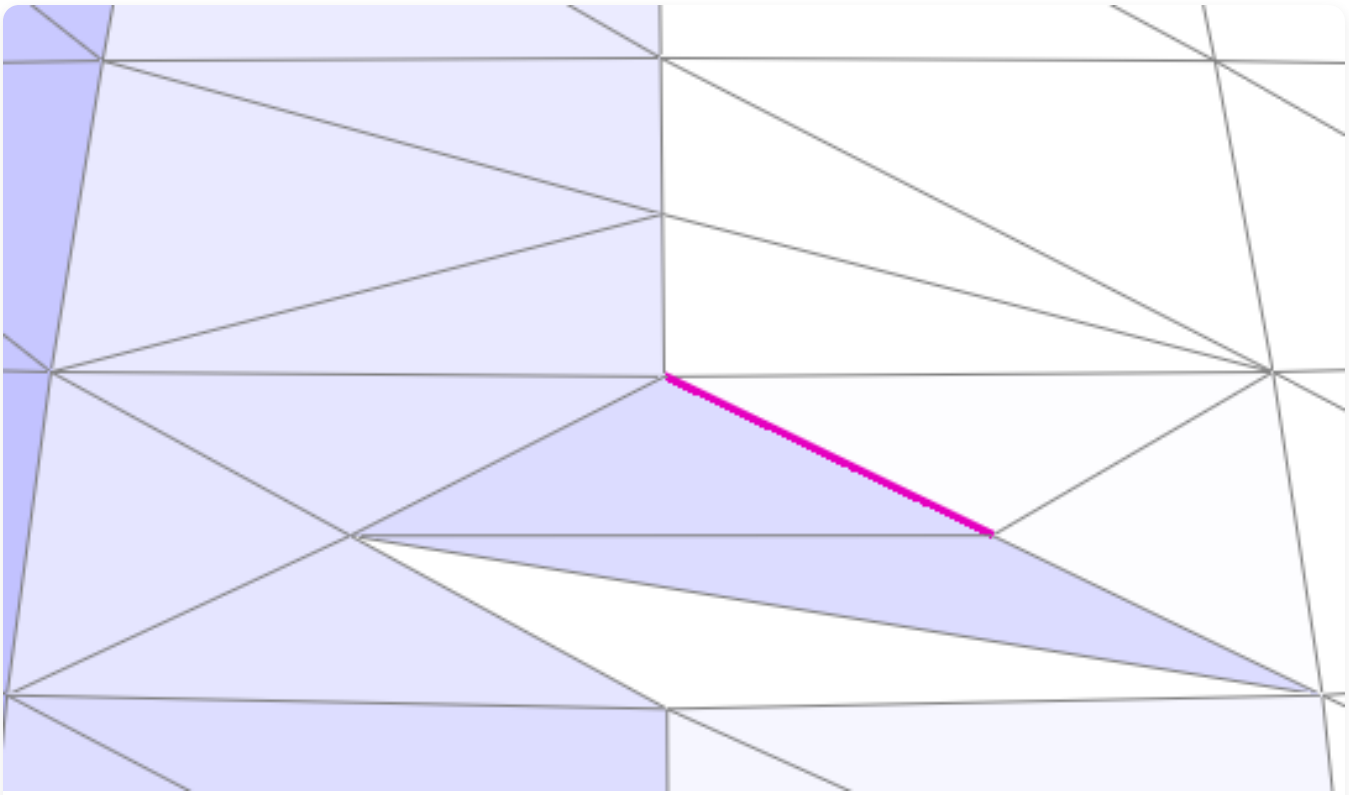
Before split



After split



Before split and flip

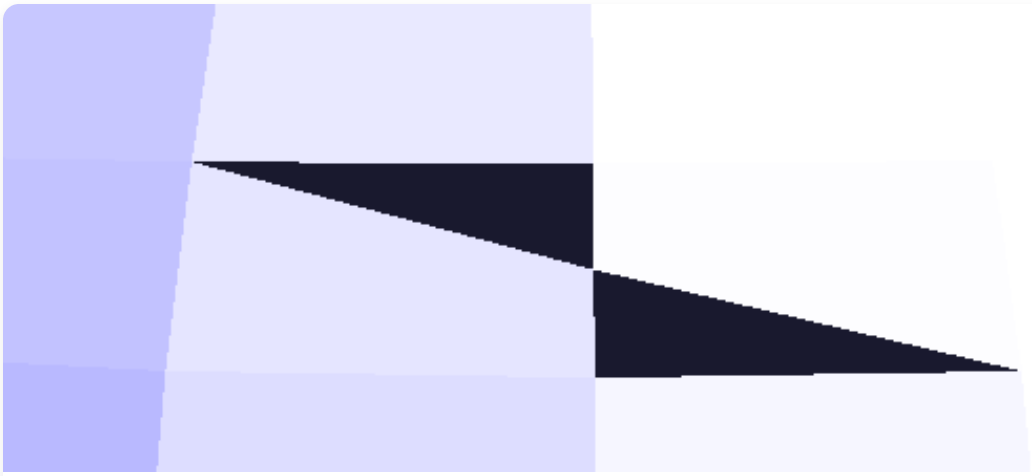


After split and flip

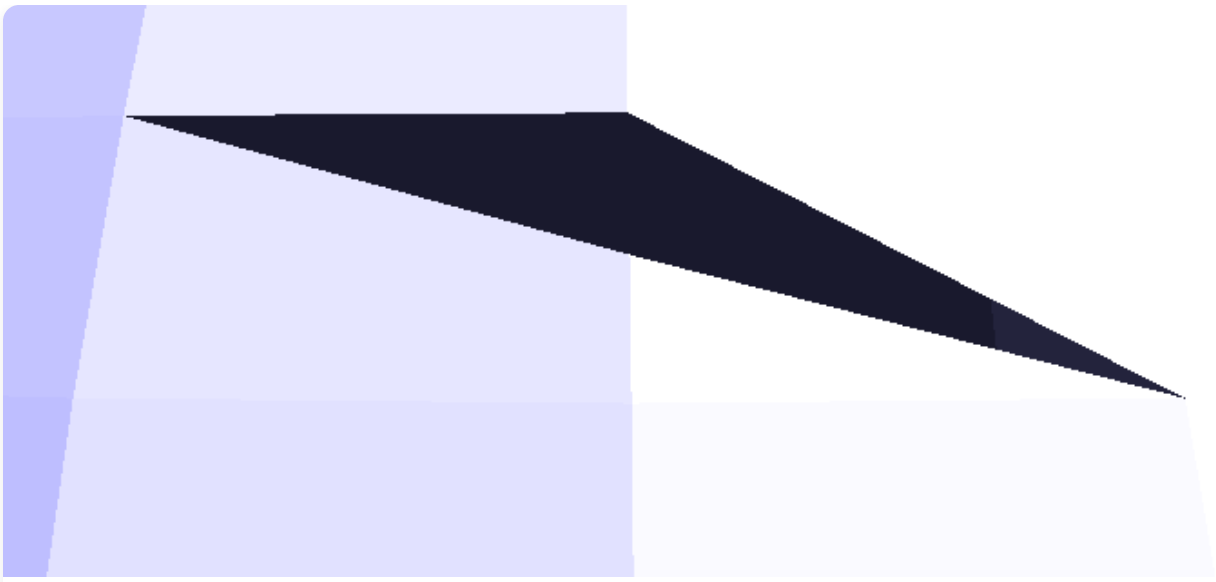
Part 6: Loop Subdivision for Mesh Upsampling

Using the weight computations defined in the spec, I looped through all edges and vertices and stored the updated positions in their `newPosition` attributes. I then collected all original edges before splitting into a vector, called split edge on each, and moved the new vertex to the saved point. To flip, I checked whether a new edge connects one old and one new vertex. Finally, I iterate through every vertex and move it to its precomputed new position.

The main observation is that sharp features get smoother with each subdivision — particularly noticeable on the teapot spout.



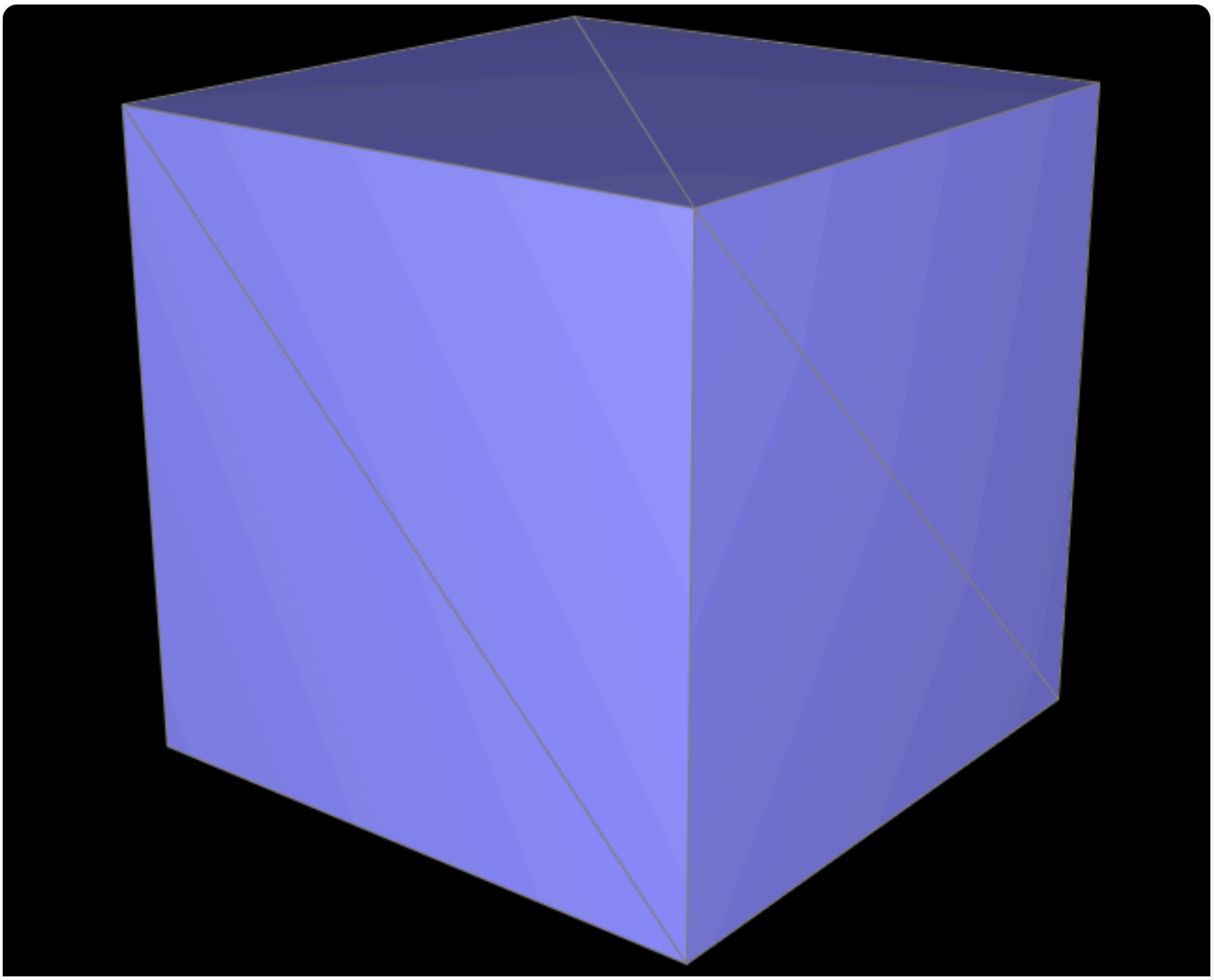
Before upsampling



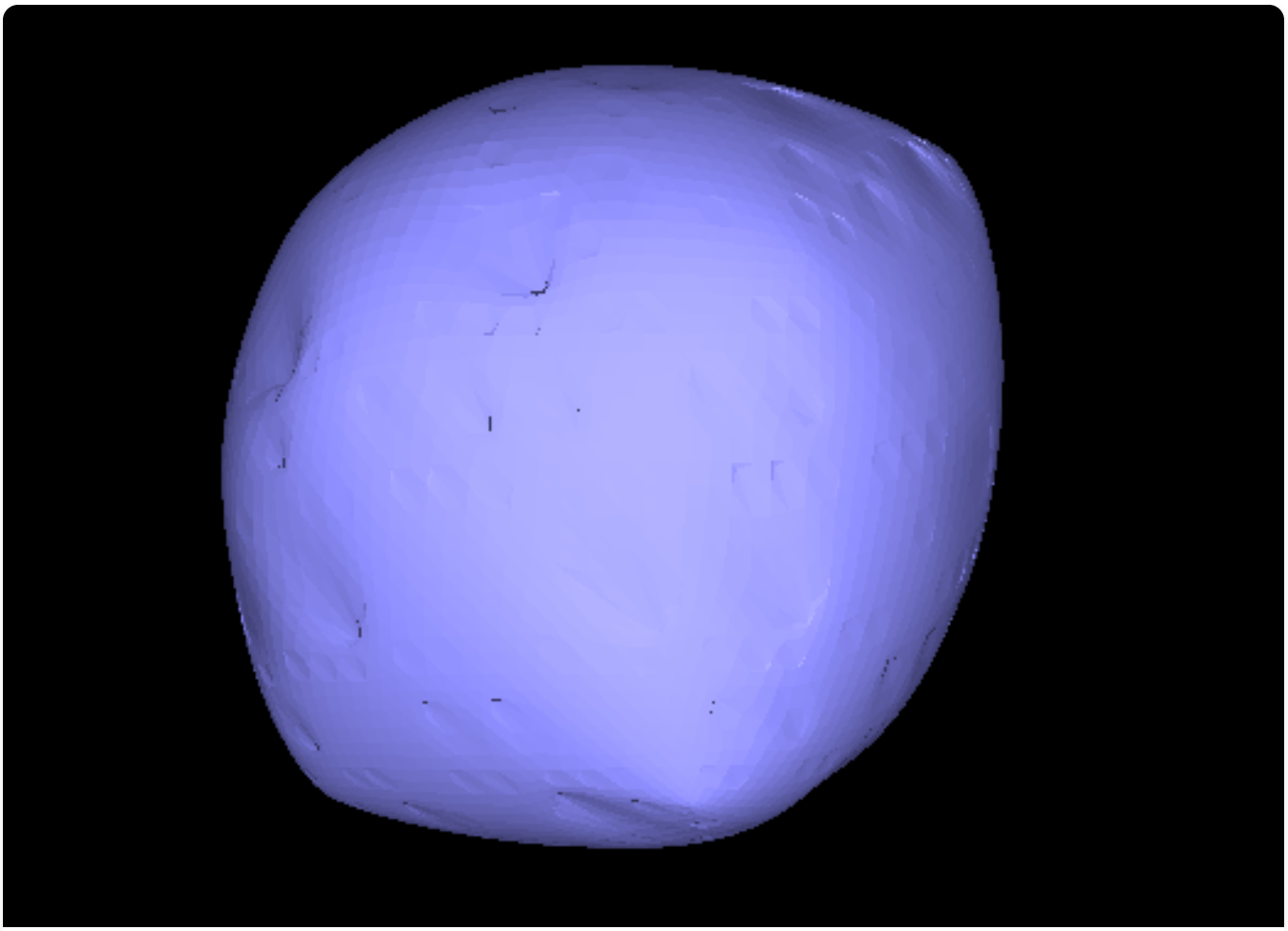
After upsampling

Cube Subdivision Experiments

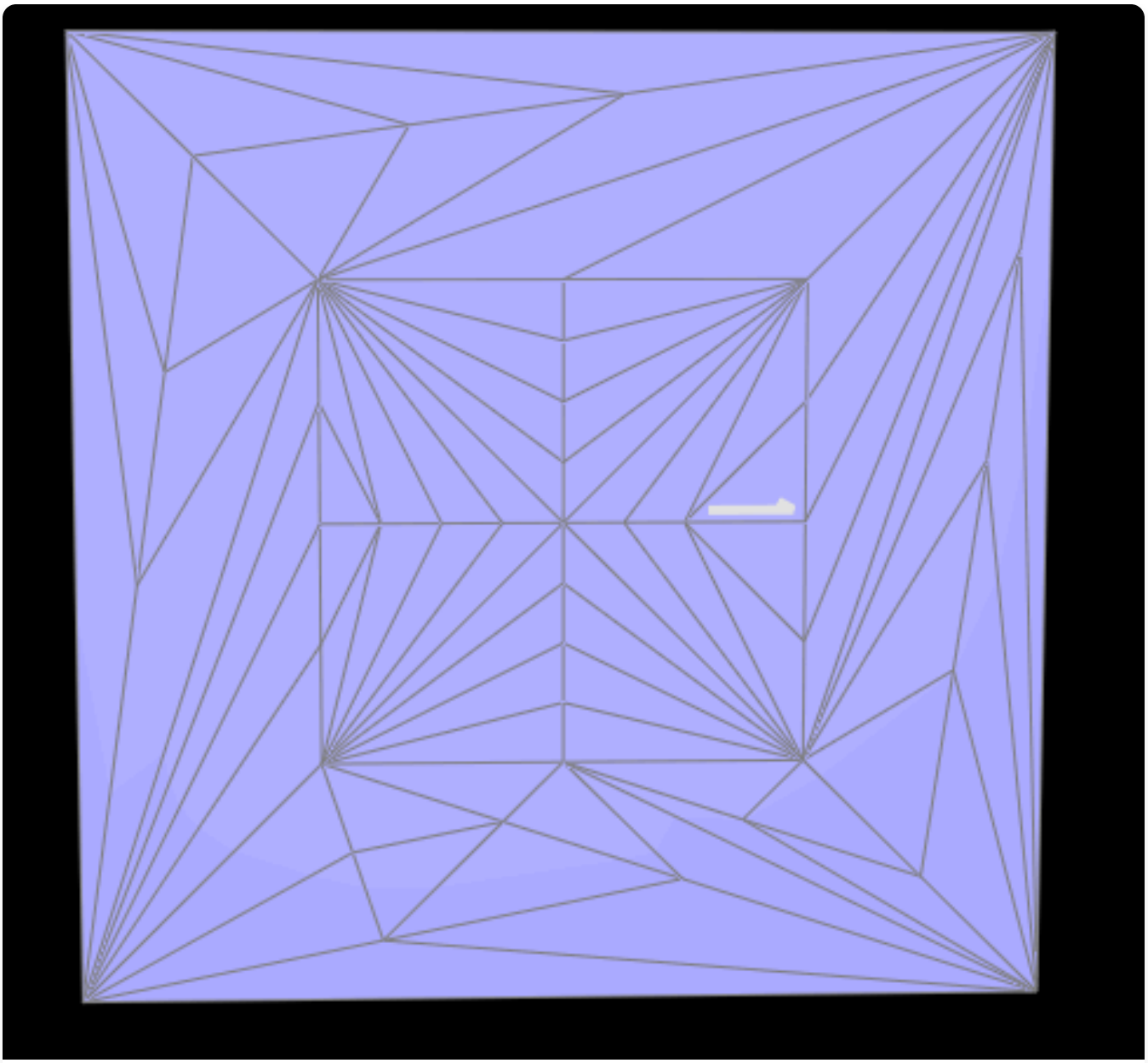
When I upsampled a cube repeatedly without any pre-processing, it became a potato shape. However, when I pre-split edges on some faces and left others alone, I noticed the pre-split faces remained more cube-like while the untouched faces rounded out more aggressively. This makes intuitive sense — more edges give the subdivision algorithm more local detail to preserve.



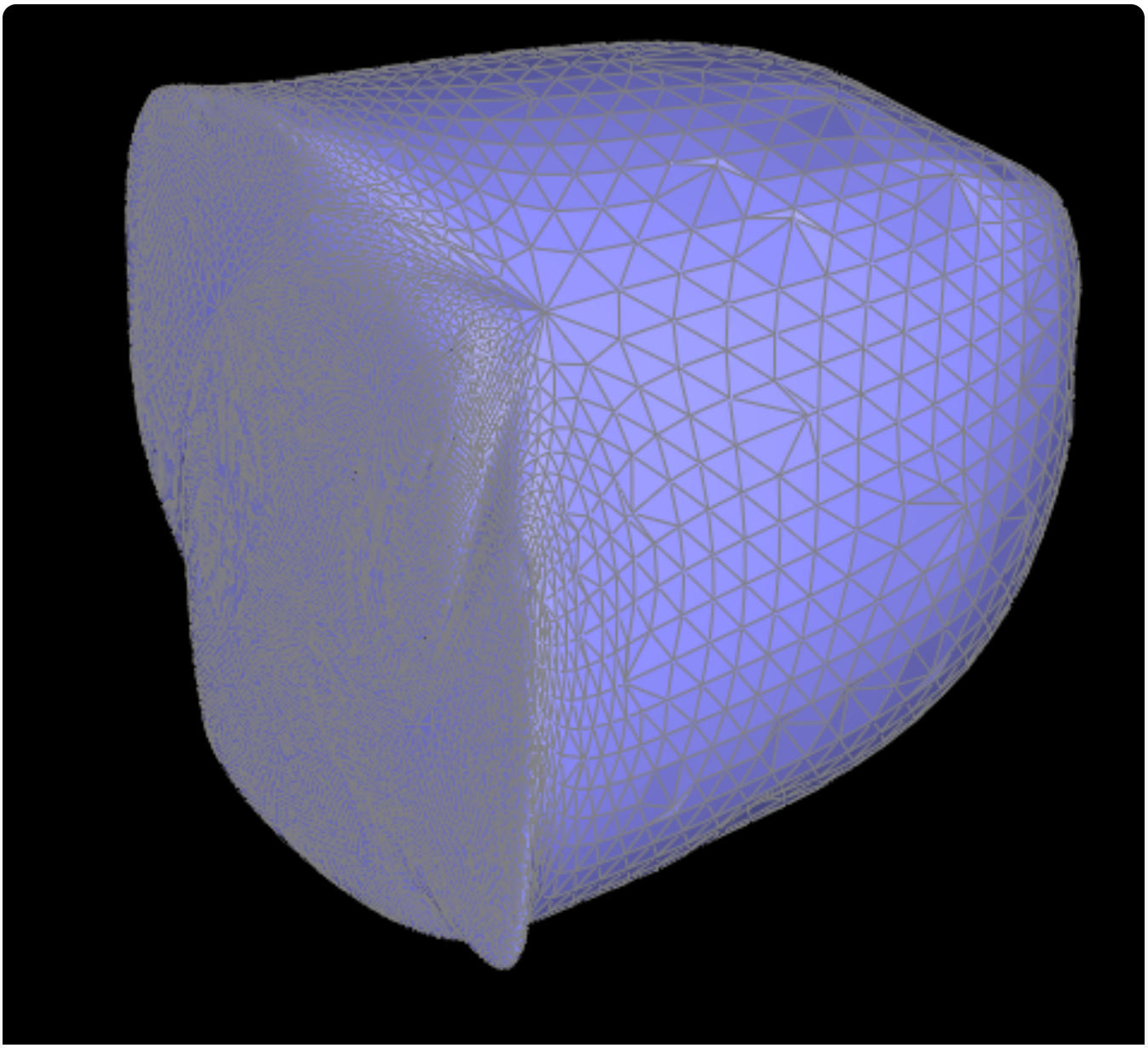
Original cube



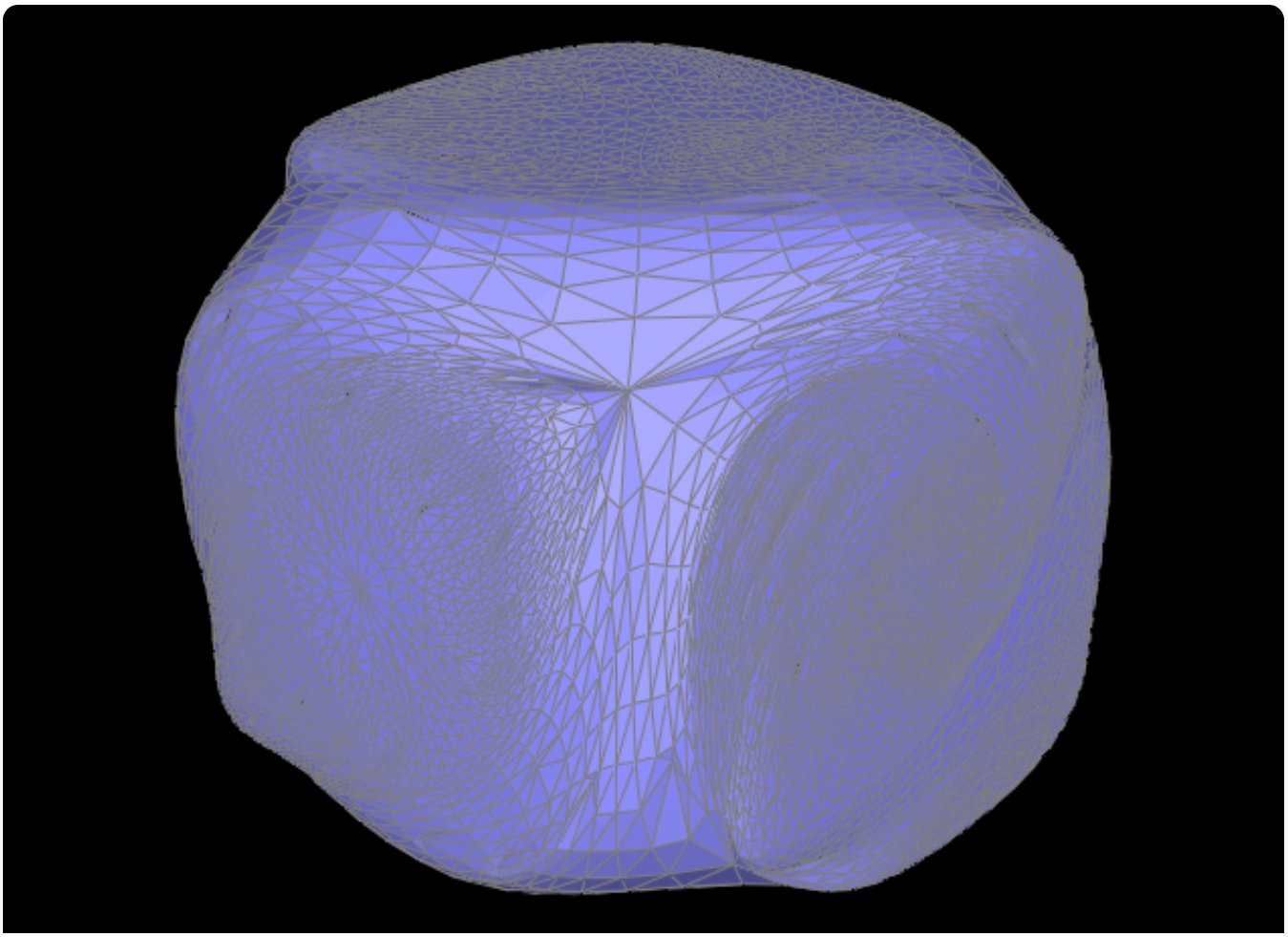
Potato (4× upsampled)



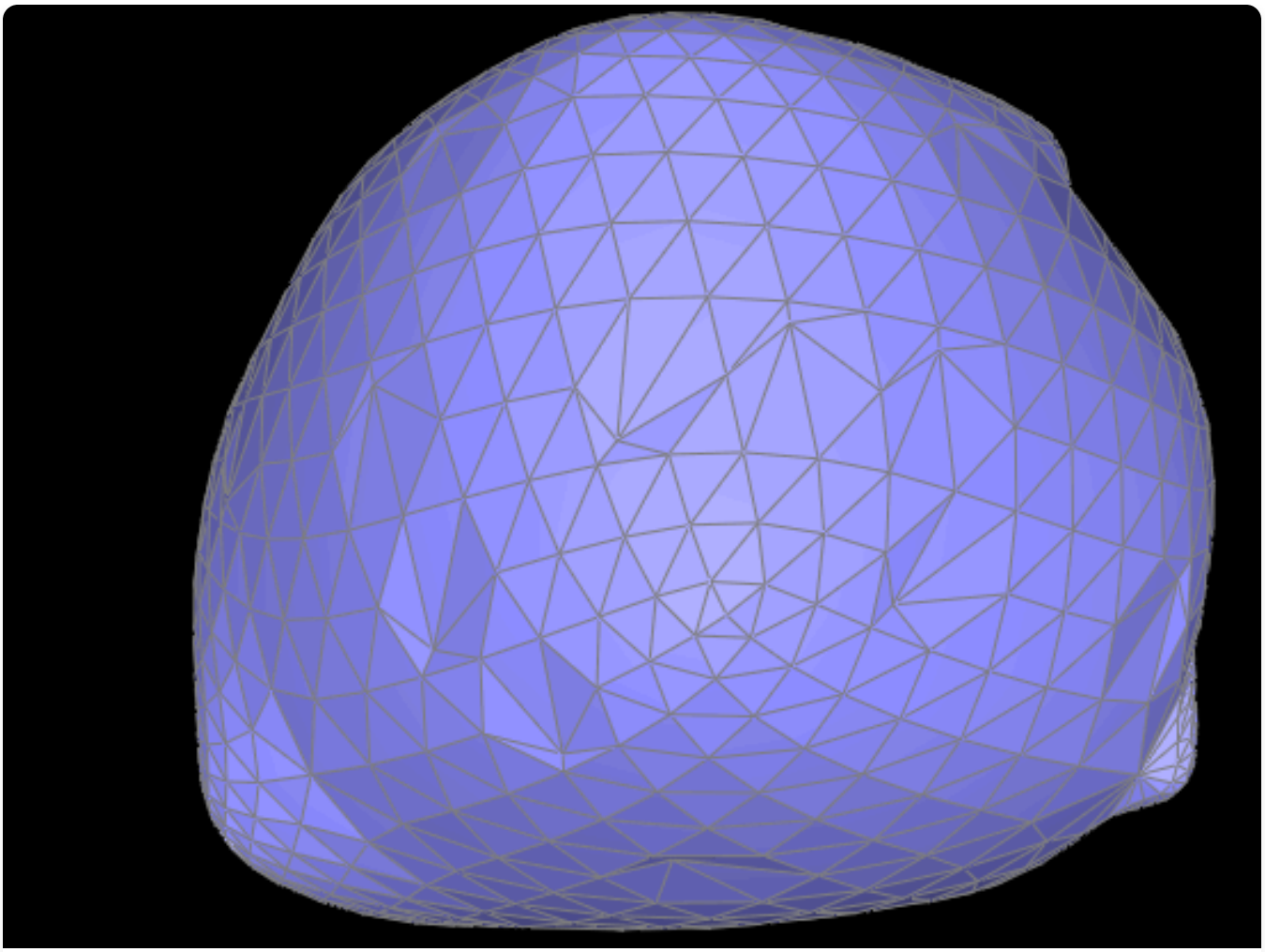
Pre-split face



Upsampling comparison



Cube-like side (pre-split)



Potato-like side (untouched)